Verification Lab

Code for the lab:
https://github.com/UCLA-Tapeout-Club/dv_assignment.git

Lab overview:

Welcome to the verification lab! This lab consists of 3 questions:

1) First word fall through FIFO + cocotb simulation
2) Formal verification
3) CDC verification + SVA

In order to complete this lab, you must complete question 1 and either question 2 **OR** question 3. If you're feeling ambitious, I would recommend doing all three questions since all three questions will be relevant to our eventual design and touch on different skills.

If you need any help with setup or completing the lab, feel free to ping Jessie (@chey0) on Discord!

Prerequisites

You will need a machine with Verilator, cocotb, gtkwaves installed for all questions and Yosys, SymbiYosys and z3 for the formal verification question. Here is a guide on how to set up all these tools for Windows, Mac, and Linux users: Verification Lab Tool Setup

Question 1: Synchronous FIFO & cocotb

The code for this question can be found in `sync_fifo/`.

Part a: Refer to `sync_fifo_1.sv` and `sync_fifo_2.sv`. Under the `waves/` folder there are two waveforms generated called `sync_fifo_1.fst` and `sync_fifo_2.fst`. Launch these waveforms using:

```
Shell
gtkwave waves/sync_fifo_1.fst
```

Answer these questions below:

What do you observe about the code and the waveforms?
Why do the modules produce different results?
How would you fix the bug present in the waveforms?

Part b:
Write a cocotb test bench to verify the provided fifo code. Please comment on coverage of your testbench (does not need to be a percentage, describe what cases you have covered and what cases you have not covered)?

Part c:
For what cases does this FIFO not work? What is one way to make this FIFO more hardware efficient? Provide a screenshot of a waveform where the FIFO behaves incorrectly.
Hint: What does this code assume during its computations?

Deliverables:

Submit the answers to the questions above (with screenshots as needed) and cocotb testbench written for part b.

Question 2: Formal Verification

Using Yosys, we will be evaluating a 4-requester round-robin arbiter.
The goal is to prove 3 properties and fix any bugs you identify.

Property A: Mutual exclusion
At most one grant may be active

Property B: Grant validity
Every grant must correspond to an active request
Hint: what are valid values for grant?

Property C: Bounded fairness
Assume a requester holds its request continuously. In this four-client, one-grant-per-cycle design, it should receive a grant within four arbitration opportunities.

The code for this question can be found in `formal_arbiter/`. The source code is in `rr_arbiter.sv`. To test each property, you will need to add code for each of the TODOs in `arbiter_formal.sv`.

Deliverables:

For each property, answer these questions:

If there was a bug, what was the bug? How did you fix the bug?

Include the code you added to `arbiter_formal.sv`

Provide a screenshot of a failing waveform with a counterexample for properties with a bug.

Question 3: CDC debugging with SVA

The code for this question can be found in `cdc_verif/`.

Part a: Refer to the synchronizer in `sync_1.sv`. Write a SVA testbench to test this synchronizer. In your testbench, specifically include a statement specifically that tests whether the output changes two cycles after the input. Is this a valid synchronizer? If it is, provide a passing waveform. If it isn't, provide a failing waveform and explain the condition under which it would fail.

Part b: Verify the synchronizer in `sync_2.sv` with a SVA testbench and provide a screenshot of results (can be terminal output or waveforms). Under what conditions does this synchronizer work?

Part c: Write a SVA testbench using the asynchronous fifo implemented in `async_fifo.sv`. The synchronizer instantiated is implemented in `sync_2.sv`. If there are bugs, list the bugs below (provide screenshots of how you caught the bug) and fix them.